Spring REST & JWT — Hands-on Answers

Doc 1 — Spring Boot Basics & Core XML Config

Hands on 1: Create Spring Web Project

```
package com.cognizant.springlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringLearnApplication {

    private static final Logger LOGGER =
LoggerFactory.getLogger(SpringLearnApplication.class);

    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
        LOGGER.info("Inside main");
    }
}
```

Hands on 2: Load SimpleDateFormat from XML

date-format.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dateFormat" class="java.text.SimpleDateFormat">
        <constructor-arg value="dd/MM/yyyy" />
    </bean>

</beans>
```

displayDate() in SpringLearnApplication.java

```
public static void displayDate() throws ParseException {
    ApplicationContext context = new ClassPathXmlApplicationContext("date-
format.xml");
    SimpleDateFormat format = context.getBean("dateFormat",
SimpleDateFormat.class);
    Date date = format.parse("31/12/2018");
    System.out.println(date);
}
```

Hands on 3: Incorporate Logging

application.properties

```
logging.level.org.springframework=info
logging.level.com.cognizant.springlearn=debug
logging.pattern.console=%d{yyMMdd}|%d{HH:mm:ss.SSS}|%-20.20thread|%5p|%-
25.25logger{25}|%25M|%m%n
```

Updated displayDate() with logging

```
private static final Logger LOGGER =
LoggerFactory.getLogger(SpringLearnApplication.class);

public static void displayDate() throws ParseException {
    LOGGER.info("START");
    ApplicationContext context = new ClassPathXmlApplicationContext("date-
format.xml");
    SimpleDateFormat format = context.getBean("dateFormat",
SimpleDateFormat.class);
    Date date = format.parse("31/12/2018");
    LOGGER.debug("{}", date);
    LOGGER.info("END");
}
```

Hands on 4: Load Country from XML

country.xml

```
<bean id="country" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN" />
    <property name="name" value="India" />
</bean>
```

Country.java

```
package com.cognizant.springlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Country {

    private static final Logger LOGGER = LoggerFactory.getLogger(Country.class);

    private String code;
    private String name;

    public Country() {
        LOGGER.debug("Inside Country Constructor");
    }

    public String getCode() {
        LOGGER.debug("getCode called");
        return code;
    }

    public void setCode(String code) {
```

```
        LOGGER.debug("setCode called with {}", code);
        this.code = code;
    }

    public String getName() {
        LOGGER.debug("getName called");
        return name;
    }

    public void setName(String name) {
        LOGGER.debug("setName called with {}", name);
        this.name = name;
    }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

displayCountry() in SpringLearnApplication.java

```
public static void displayCountry() {
    LOGGER.info("START");
    ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml");
    Country country = context.getBean("country", Country.class);
    LOGGER.debug("Country : {}", country.toString());
    LOGGER.info("END");
}

public static void main(String[] args) {
    SpringApplication.run(SpringLearnApplication.class, args);
    displayCountry();
}
```

Hands on 5: Singleton vs Prototype Scope

Singleton demo

```
ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
Country country = context.getBean("country", Country.class);
Country anotherCountry = context.getBean("country", Country.class);
// constructor is called only once - both references point to the same object
```

Prototype demo — country.xml

```
<bean id="country" class="com.cognizant.springlearn.Country" scope="prototype">
    <property name="code" value="IN" />
    <property name="name" value="India" />
</bean>
<!-- constructor now called twice - two different objects created -->
```

Hands on 6: Load List of Countries from XML

country.xml

```
<bean id="in" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN" />
    <property name="name" value="India" />
</bean>
<bean id="us" class="com.cognizant.springlearn.Country">
    <property name="code" value="US" />
    <property name="name" value="United States" />
</bean>
<bean id="de" class="com.cognizant.springlearn.Country">
    <property name="code" value="DE" />
    <property name="name" value="Germany" />
</bean>
<bean id="jp" class="com.cognizant.springlearn.Country">
    <property name="code" value="JP" />
    <property name="name" value="Japan" />
</bean>

<bean id="countryList" class="java.util.ArrayList">
    <constructor-arg>
        <list>
            <ref bean="in"></ref>
            <ref bean="us"></ref>
            <ref bean="de"></ref>
            <ref bean="jp"></ref>
        </list>
    </constructor-arg>
</bean>
```

displayCountries() in SpringLearnApplication.java

```
@SuppressWarnings("unchecked")
public static void displayCountries() {
    LOGGER.info("START");
    ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml");
    List<Country> countryList = (List<Country>) context.getBean("countryList");
    LOGGER.debug("Countries : {}", countryList);
    LOGGER.info("END");
}
```

Doc 2 — RESTful GET Services & MockMVC

Hello World REST Service

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private static final Logger LOGGER =
LoggerFactory.getLogger(HelloController.class);

    @GetMapping("/hello")
    public String sayHello() {
        LOGGER.info("START");
        LOGGER.info("END");
        return "Hello World!!";
    }
}
```

REST — Country Web Service (single country)

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.springlearn.Country;

@RestController
public class CountryController {

    private static final Logger LOGGER =
LoggerFactory.getLogger(CountryController.class);

    @RequestMapping("/country")
    public Country getCountryIndia() {
        LOGGER.info("START");
        ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("in", Country.class);
        LOGGER.info("END");
        return country;
    }
}
```

REST — Get All Countries

```
@GetMapping("/countries")
public List<Country> getAllCountries() {
    LOGGER.info("START");
    ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml");
    List<Country> countryList = (List<Country>) context.getBean("countryList");
    LOGGER.info("END");
    return countryList;
}
```

REST — Get Country by Code

CountryController.java

```
@GetMapping("/countries/{code}")
public Country getCountry(@PathVariable String code) throws
CountryNotFoundException {
    LOGGER.info("START");
    Country country = countryService.getCountry(code);
    LOGGER.info("END");
    return country;
}
```

CountryService.java

```
@Service
public class CountryService {

    private static final Logger LOGGER =
LoggerFactory.getLogger(CountryService.class);

    public Country getCountry(String code) throws CountryNotFoundException {
        LOGGER.info("START");
        ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml");
        List<Country> countryList = (List<Country>)
context.getBean("countryList");

        Country result = countryList.stream()
                .filter(c -> c.getCode().equalsIgnoreCase(code))
                .findFirst()
                .orElseThrow(() -> new CountryNotFoundException());

        LOGGER.info("END");
        return result;
    }
}
```

REST — Get Country Exceptional Scenario

```
package com.cognizant.springlearn.service.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;
```

```
@ResponseStatus(value = HttpStatus.NOT_FOUND, reason = "Country not found")
public class CountryNotFoundException extends Exception {
}
// curl -i http://localhost:8090/country/az
// gives 404 with "Country not found" reason
```

MockMVC — Test Get Country Service

```
package com.cognizant.springlearn;

import static org.junit.Assert.assertNotNull;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.junit4.SpringRunner;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;
import com.cognizant.springlearn.controller.CountryController;

@RunWith(SpringRunner.class)
@SpringBootTest
@AutoConfigureMockMvc
public class SpringLearnApplicationTests {

    @Autowired
    private CountryController countryController;

    @Autowired
    private MockMvc mvc;

    @Test
    public void contextLoads() {
        assertNotNull(countryController);
    }

    @Test
    public void getCountry() throws Exception {
        ResultActions actions = mvc.perform(get("/country"));
        actions.andExpect(status().isOk());
        actions.andExpect(jsonPath("$.code").exists());
        actions.andExpect(jsonPath("$.code").value("IN"));
        actions.andExpect(jsonPath("$.name").exists());
        actions.andExpect(jsonPath("$.name").value("India"));
    }

    @Test
```

```
    public void testGetCountryException() throws Exception {
        ResultActions actions = mvc.perform(get("/countries/az"));
        actions.andExpect(status().isNotFound());
        actions.andExpect(status().reason("Country not found"));
    }
}
```

Doc 3 — Employee & Department GET Services

Static Employee List (employee.xml) + EmployeeDao

```
<bean id="dept1" class="com.cognizant.springlearn.model.Department">
    <property name="id" value="1" />
    <property name="name" value="Engineering" />
</bean>
<bean id="dept2" class="com.cognizant.springlearn.model.Department">
    <property name="id" value="2" />
    <property name="name" value="HR" />
</bean>

<bean id="emp1" class="com.cognizant.springlearn.model.Employee">
    <property name="id" value="1" />
    <property name="name" value="Ravi Kumar" />
    <property name="salary" value="45000" />
    <property name="department" ref="dept1" />
</bean>
<bean id="emp2" class="com.cognizant.springlearn.model.Employee">
    <property name="id" value="2" />
    <property name="name" value="Anita Sharma" />
    <property name="salary" value="52000" />
    <property name="department" ref="dept2" />
</bean>
<!-- emp3, emp4 similarly -->

<bean id="employeeList" class="java.util.ArrayList">
    <constructor-arg>
        <list>
            <ref bean="emp1" />
            <ref bean="emp2" />
            <ref bean="emp3" />
            <ref bean="emp4" />
        </list>
    </constructor-arg>
</bean>
```

```
@Repository
public class EmployeeDao {

    private static final Logger LOGGER =
LoggerFactory.getLogger(EmployeeDao.class);
    private static ArrayList<Employee> EMPLOYEE_LIST;

    @SuppressWarnings("unchecked")
    public EmployeeDao() {
        ApplicationContext context = new
ClassPathXmlApplicationContext("employee.xml");
        EMPLOYEE_LIST = (ArrayList<Employee>) context.getBean("employeeList");
    }

    public ArrayList<Employee> getAllEmployees() {
        LOGGER.info("START");
        LOGGER.info("END");
        return EMPLOYEE_LIST;
```

```
    }
}
```

REST Service to Get All Employees

```
@Service
public class EmployeeService {

    private static final Logger LOGGER =
LoggerFactory.getLogger(EmployeeService.class);

    @Autowired
    private EmployeeDao employeeDao;

    @Transactional
    public List<Employee> getAllEmployees() {
        LOGGER.info("START");
        List<Employee> employees = employeeDao.getAllEmployees();
        LOGGER.info("END");
        return employees;
    }
}
@RestController
public class EmployeeController {

    private static final Logger LOGGER =
LoggerFactory.getLogger(EmployeeController.class);

    @Autowired
    private EmployeeService employeeService;

    @GetMapping("/employees")
    public List<Employee> getAllEmployees() {
        LOGGER.info("START");
        List<Employee> employees = employeeService.getAllEmployees();
        LOGGER.info("END");
        return employees;
    }
}
```

REST Service for Department

```
@Repository
public class DepartmentDao {

    private static ArrayList<Department> DEPARTMENT_LIST;

    @SuppressWarnings("unchecked")
    public DepartmentDao() {
        ApplicationContext context = new
ClassPathXmlApplicationContext("employee.xml");
        DEPARTMENT_LIST = (ArrayList<Department>)
context.getBean("departmentList");
    }

    public ArrayList<Department> getAllDepartments() {
```

```
        return DEPARTMENT_LIST;
    }
}

@Service
public class DepartmentService {

    @Autowired
    private DepartmentDao departmentDao;

    public List<Department> getAllDepartments() {
        return departmentDao.getAllDepartments();
    }
}

@RestController
public class DepartmentController {

    private static final Logger LOGGER =
LoggerFactory.getLogger(DepartmentController.class);

    @Autowired
    private DepartmentService departmentService;

    @GetMapping("/departments")
    public List<Department> getAllDepartments() {
        LOGGER.info("START");
        List<Department> departments = departmentService.getAllDepartments();
        LOGGER.info("END");
        return departments;
    }
}
```

Doc 4 — POST / PUT / DELETE with Validation

CountryController with URL Convention

```
@RestController
@RequestMapping("/countries")
public class CountryController {

    private static final Logger LOGGER =
LoggerFactory.getLogger(CountryController.class);

    @Autowired
    private CountryService countryService;

    @GetMapping
    public List<Country> getAllCountries() {
        return countryService.getAllCountries();
    }

    @GetMapping("/{code}")
    public Country getCountry(@PathVariable String code) throws
CountryNotFoundException {
        return countryService.getCountry(code);
    }

    @PostMapping
    public void addCountry() {
        LOGGER.info("Start");
    }
}
```

Read Country as Request Body

```
@PostMapping
public Country addCountry(@RequestBody Country country) {
    LOGGER.info("Start");
    LOGGER.debug("Country:{}", country);
    return country;
}
curl -i -H 'Content-Type: application/json' -X POST -s \
  -d '{"code":"IN","name":"India"}' http://localhost:8090/countries
```

Validating Country Code

Country.java — validation annotations

```
@NotNull
@Size(min = 2, max = 2, message = "Country code should be 2 characters")
private String code;
```

Manual validation in controller (first pass)

```
ValidatorFactory factory = Validation.buildDefaultValidatorFactory();
Validator validator = factory.getValidator();
```

```
Set<ConstraintViolation<Country>> violations = validator.validate(country);
List<String> errors = new ArrayList<String>();
for (ConstraintViolation<Country> violation : violations) {
    errors.add(violation.getMessage());
}
if (violations.size() > 0) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST, errors.toString());
}
```

Global Exception Handler

```
package com.cognizant.springlearn;

import java.util.ArrayList;
import java.util.Date;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import
org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHand
ler;

@ControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    private static final Logger LOGGER =
LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex, HttpHeaders headers,
            HttpStatus status, WebRequest request) {

        LOGGER.info("Start");

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", new Date());
        body.put("status", status.value());

        List<String> errors = ex.getBindingResult().getFieldErrors().stream()
                .map(x -> x.getDefaultMessage())
                .collect(Collectors.toList());
        body.put("errors", errors);

        LOGGER.info("End");
        return new ResponseEntity<>(body, headers, status);
    }
}
```

addCountry() now uses @Valid instead of manual checks

```
@PostMapping
public Country addCountry(@RequestBody @Valid Country country) {
    LOGGER.info("Start");
    LOGGER.debug("Country:{}", country);
    return country;
}
```

Handle Number Format Errors (Global Handler)

```
@Override
protected ResponseEntity<Object> handleHttpMessageNotReadable(
        HttpMessageNotReadableException ex, HttpHeaders headers,
        HttpStatus status, WebRequest request) {

    Map<String, Object> body = new LinkedHashMap<>();
    body.put("timestamp", new Date());
    body.put("status", status.value());
    body.put("error", "Bad Request");

    if (ex.getCause() instanceof InvalidFormatException) {
        final Throwable cause = ex.getCause() == null ? ex : ex.getCause();
        for (InvalidFormatException.Reference reference :
((InvalidFormatException) cause).getPath()) {
            body.put("message", "Incorrect format for field '" +
reference.getFieldName() + "'");
        }
    }

    return new ResponseEntity<>(body, headers, status);
}
```

Employee, Department, Skill Validations

```
// Employee.java
@NotNull
private Integer id;

@NotNull
@NotBlank
@Size(min = 1, max = 30)
private String name;

@NotNull
@Min(0)
private Double salary;

@NotNull
private Boolean permanent;

@JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "dd/MM/yyyy")
private Date dateOfBirth;

// Department.java / Skill.java
@NotNull
```

```
private Integer id;

@NotNull
@NotBlank
@Size(min = 1, max = 30)
private String name;
```

REST Service to Update Employee

```
@ResponseStatus(value = HttpStatus.NOT_FOUND, reason = "Employee not found")
public class EmployeeNotFoundException extends Exception {
}
```

```
// EmployeeDao.java
public void updateEmployee(Employee employee) throws EmployeeNotFoundException {
    boolean found = false;
    for (int i = 0; i < EMPLOYEE_LIST.size(); i++) {
        if (EMPLOYEE_LIST.get(i).getId().equals(employee.getId())) {
            EMPLOYEE_LIST.set(i, employee);
            found = true;
            break;
        }
    }
    if (!found) {
        throw new EmployeeNotFoundException();
    }
}
```

```
// EmployeeService.java
@Transactional
public void updateEmployee(Employee employee) throws EmployeeNotFoundException {
    employeeDao.updateEmployee(employee);
}
```

```
// EmployeeController.java
@PutMapping
public void updateEmployee(@RequestBody @Valid Employee employee) throws
EmployeeNotFoundException {
    LOGGER.info("Start");
    employeeService.updateEmployee(employee);
    LOGGER.info("End");
}
```

REST DELETE Service

```
// EmployeeDao.java
public void deleteEmployee(Integer id) throws EmployeeNotFoundException {
    boolean removed = EMPLOYEE_LIST.removeIf(e -> e.getId().equals(id));
    if (!removed) {
        throw new EmployeeNotFoundException();
    }
}

// EmployeeService.java
@Transactional
public void deleteEmployee(Integer id) throws EmployeeNotFoundException {
    employeeDao.deleteEmployee(id);
}
```

```
// EmployeeController.java
@DeleteMapping("/{id}")
public void deleteEmployee(@PathVariable Integer id) throws
EmployeeNotFoundException {
    LOGGER.info("Start");
    employeeService.deleteEmployee(id);
    LOGGER.info("End");
}
```

Doc 5 — JWT Authentication with Spring Security

Secure REST Services with Spring Security

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
package com.cognizant.springlearn.security;

import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurit
y;
import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfi
gurerAdapter;

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
}
```

Creating Users and Roles

```
package com.cognizant.springlearn.security;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.authentication.builders.Authentica
tionManagerBuilder;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurit
y;
import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfi
gurerAdapter;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    private static final Logger LOGGER =
LoggerFactory.getLogger(SecurityConfig.class);

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception
{
```

```
        auth.inMemoryAuthentication()
            .withUser("admin").password(passwordEncoder().encode("pwd")).roles("A
DMIN")
            .and()
            .withUser("user").password(passwordEncoder().encode("pwd")).roles("US
ER");
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        LOGGER.info("Start");
        return new BCryptPasswordEncoder();
    }

    @Override
    protected void configure(HttpSecurity httpSecurity) throws Exception {
        httpSecurity.csrf().disable().httpBasic().and()
            .authorizeRequests().antMatchers("/countries").hasRole("USER");
    }
}
```

```
# valid
curl -s -u user:pwd http://localhost:8090/countries

# wrong password -> 401 Unauthorized
curl -s -u user:pwd1 http://localhost:8090/countries

# wrong role -> 403 Forbidden
curl -s -u admin:pwd http://localhost:8090/countries
```

Authentication Controller — Return JWT

AuthenticationController.java (basic skeleton)

```
package com.cognizant.springlearn.controller;

import java.util.HashMap;
import java.util.Map;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AuthenticationController {

    private static final Logger LOGGER =
LoggerFactory.getLogger(AuthenticationController.class);

    @GetMapping("/authenticate")
    public Map<String, String> authenticate(@RequestHeader("Authorization")
String authHeader) {
        LOGGER.info("Start");
        LOGGER.debug("authHeader:{}", authHeader);

        Map<String, String> map = new HashMap<>();
        map.put("token", "");
```

```
        LOGGER.info("End");
        return map;
    }
}
```

SecurityConfig.java — allow /authenticate for both roles

```
.antMatchers("/countries").hasRole("USER")
.antMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
```

Decode Username from Authorization Header

```
private String getUser(String authHeader) {
    LOGGER.info("Start");
    String encodedCredentials = authHeader.replace("Basic ", "");
    byte[] decodedBytes = Base64.getDecoder().decode(encodedCredentials);
    String decodedString = new String(decodedBytes);
    String user = decodedString.substring(0, decodedString.indexOf(":"));
    LOGGER.debug("user:{}", user);
    LOGGER.info("End");
    return user;
}
@GetMapping("/authenticate")
public Map<String, String> authenticate(@RequestHeader("Authorization") String
authHeader) {
    LOGGER.info("Start");
    String user = getUser(authHeader);

    Map<String, String> map = new HashMap<>();
    map.put("token", "");

    LOGGER.info("End");
    return map;
}
```

Generate JWT Token

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.0</version>
</dependency>
private String generateJwt(String user) {
    LOGGER.info("Start");
    JwtBuilder builder = Jwts.builder();
    builder.setSubject(user);
    builder.setIssuedAt(new Date());
    builder.setExpiration(new Date((new Date()).getTime() + 1200000));
    builder.signWith(SignatureAlgorithm.HS256, "secretkey");
    String token = builder.compact();
    LOGGER.info("End");
    return token;
}
```

Wiring it into authenticate()

```
@GetMapping("/authenticate")
public Map<String, String> authenticate(@RequestHeader("Authorization") String
authHeader) {
    LOGGER.info("Start");
    String user = getUser(authHeader);
    String token = generateJwt(user);

    Map<String, String> map = new HashMap<>();
    map.put("token", token);

    LOGGER.info("End");
    return map;
}
```

Authorize Requests using JWT Filter

```
package com.cognizant.springlearn.security;

import java.io.IOException;
import java.util.ArrayList;
import javax.servlet.FilterChain;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import
org.springframework.security.web.authentication.www.BasicAuthenticationFilter;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jws;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;

public class JwtAuthorizationFilter extends BasicAuthenticationFilter {

    private static final Logger LOGGER =
LoggerFactory.getLogger(JwtAuthorizationFilter.class);

    public JwtAuthorizationFilter(AuthenticationManager authenticationManager) {
        super(authenticationManager);
        LOGGER.info("Start");
        LOGGER.debug("{}: ", authenticationManager);
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse
res,
            FilterChain chain) throws IOException, ServletException {

        LOGGER.info("Start");
        String header = req.getHeader("Authorization");
```

```
        LOGGER.debug(header);

        if (header == null || !header.startsWith("Bearer ")) {
            chain.doFilter(req, res);
            return;
        }

        UsernamePasswordAuthenticationToken authentication =
getAuthentication(req);
        SecurityContextHolder.getContext().setAuthentication(authentication);
        chain.doFilter(req, res);
        LOGGER.info("End");
    }

    private UsernamePasswordAuthenticationToken
getAuthentication(HttpServletRequest request) {
        String token = request.getHeader("Authorization");
        if (token != null) {
            try {
                Jws<Claims> jws = Jwts.parser()
                        .setSigningKey("secretkey")
                        .parseClaimsJws(token.replace("Bearer ", ""));

                String user = jws.getBody().getSubject();
                LOGGER.debug(user);

                if (user != null) {
                    return new UsernamePasswordAuthenticationToken(user, null,
new ArrayList<>());
                }
            } catch (JwtException ex) {
                return null;
            }
        }
        return null;
    }
}
```

SecurityConfig.java — plug in the filter

```
@Override
protected void configure(HttpSecurity httpSecurity) throws Exception {
    httpSecurity.csrf().disable().httpBasic().and()
        .authorizeRequests()
        .antMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
        .anyRequest().authenticated()
        .and()
        .addFilter(new JwtAuthorizationFilter(authenticationManager()));
}
```

Testing

```
# get a token
curl -s -u user:pwd http://localhost:8090/authenticate

# use the token
curl -s -H "Authorization: Bearer REPLACE_TOKEN_HERE"
http://localhost:8090/countries
```